



DOSSIER DE PRÉSENTATION · V1.0 · SEPTEMBRE 2026

MOTO Editor

Présentation détaillée du projet

IDE 100 % local, sans cloud ni Electron, construit « fait maison » par un fondateur solo assisté de trois IA. Architecture Zed, interface minimaliste type Claude Code, complexité portée par le backend et 100+ paramètres. Document objectif : acquis réels, limites assumées, feuille de route.

Stack

.NET 10 · MAUI · WinUI 3

Projets

Engine · Core · Editor

Builds

0 erreur (3 projets)

Modèle

Open source + freemium

Statut

Pré-v1.0 (bêta interne)

Sommaire

1. Synthèse exécutive & fiche d'identité	p.3
2. Genèse & méthode « 1 humain + 3 IA »	p.4
3. Doctrine & vision produit	p.5
4. Architecture & modules	p.6
5. Stack technique & contraintes	p.7
6. L'éditeur : cœur de métier	p.8
7. Panneaux modulaires façon Zed	p.9
8. IA intégrée, agents & moteurs internes	p.10-11
9. Sécurité, mises à jour & privacy	p.12
10. Performance & optimisations	p.13
11. Réglages : 100+ paramètres	p.14
12. Collab, remote, export & docs	p.15
13. Design & identité visuelle	p.16
14. Marché & comparaison concurrentielle	p.17
15. SWOT & positionnement	p.18
16. Roadmap & modèle économique	p.19
17. Limites honnêtes, méthode & conclusion	p.20

Fiche d'identité

Champ	Valeur
Nom / dépôt	MOTO Editor — github.com/Tomoushie/MOTO-Editor (public, branche main)
Nature	IDE / éditeur de code 100 % local ; aucun cloud ; aucune dépendance runtime externe
Stack	.NET 10 (net10.0-windows10.0.19041.0), C#, MAUI, WinUI 3, Windows App SDK 1.8 épinglé
Projets	Snake2000.Engine (XENO) · Moto.Core (374 fichiers) · Moto.Editor
Équipe	1 fondateur (non-développeur de métier) + 3 IA (Claude Code = repo, Qwen & Copilot = conseil)
Références	Zed (architecture), Claude Code (UX), Zen (menus IA), VS Code (richesse fonctionnelle)
Baseline	« Léger. Rapide. Sans dépendance. Tout fait maison. »

1. Synthèse exécutive

MOTO Editor est un environnement de développement **entièrement local** qui reproduit les capacités d'un IDE moderne de type Zed — docks modulaires, terminal, IA intégrée, collaboration — avec une **interface minimaliste** inspirée de Claude Code/ChatGPT et une complexité déplacée vers le backend et vers plus de 100 paramètres de configuration. Le projet est développé « fait maison » : aucun framework UI tiers, aucun service cloud, aucune télémétrie.

3 PROJETS · 0 ERREUR BUILD	374 FICHIERS MOTO.CORE	~300 PARAMÈTRES DATA-DRIVEN	20 AGENTS IA SPÉCIALISÉS
5 BRIQUES PANNEAUX ZED LIVRÉES	13+ MOTEURS INTERNES	9 FORMATS D'EXPORT	1 PLATEFORME (WINDOWS, AUJ.)

Ce que le projet est (et n'est pas)

✓ C'est

- Un éditeur/IDE local-first : édite, affiche, ouvre, sauvegarde, exécute des commandes.
- Un shell modulaire (3 docks) avec terminal interactif réel et glisser-déposer.
- Une IA locale (Ollama/ONNX) assistée d'un pipeline structuré XENO-SSS[∞].
- Un chantier transparent : bugs publics, roadmap ouverte, méthode documentée.

⚠ Ce n'est (volontairement) pas

- Pas un produit cloud : aucune donnée ne quitte la machine.
- Pas un clone de code : inspiration Zed/VS Code, réécriture C# intégrale (Zed = GPL).
- Pas encore multiplateforme : Windows d'abord, portages planifiés (PlatformEngine).
- Pas une v1.0 : LSP Roslyn, CRDT réel, DAP, tests, i18n, accessibilité = « à venir ».

2. Genèse & méthode « 1 humain + 3 IA »

Le projet est né d'un constat simple : les IDE modernes sont lourds, cloud-dépendants ou verrouillés. Le fondateur, **sans formation de développeur**, a choisi une méthode inhabituelle : orchestrer trois IA complémentaires pour construire un vrai produit, en assumant publiquement ce fonctionnement (build in public).

Rôle	IA	Périmètre réel
Exécution	Claude Code	Accès direct au dépôt ; applique, vérifie contre le vrai code, committe, capture, publie MSIX.
Architecture	Qwen	Conseil, diagnostics, documents stratégiques (marché, SWOT, roadmap, budget) ; aucun accès repo.
Conseil	Copilot	Analyses structurelles, contexte industrie.
Décision	Tom (fondateur)	Arbitre final ; teste visuellement ; fixe la doctrine et les priorités.

La réhabilitation (état initial → état actuel)

Constat initial	Action	Résultat
Plusieurs sessions IA avaient écrit des morceaux sans jamais les assembler ni compiler	Lecture de 44-47 fichiers de doc, vérification de chaque affirmation sur disque, activation Git + instantané	Base saine, annulable
Moto.Core : 374 fichiers jamais réunis, cœurs manquants	Reconstruction du csproj + moteurs manquants	0 erreur, commité
MainPage.xaml réduite à 16 lignes, abréviation XAML non déclarée	Reconstruction de la page principale	Shell fonctionnel
.NET 8 visé mais absent de la machine	Bascule .NET 10 ; NuGet débloqué par config locale	Builds verts
~15 fichiers fantômes à la racine	Tri, dossiers, nettoyage progressif	Dépôt lisible

Discipline de travail installée

Règles d'or (12)

- Additif uniquement ; jamais de suppression de feature.
- Code complet, compilable, méthodes < 40 lignes, XML docs.
- Namespaces Microsoft.UI.Xaml pour le natif ; #if WINDOWS.
- Cohérence MotoTheme.xaml ; impacts architecturaux expliqués.

Garde-fous

- Revue croisée avant commit pour tout changement structurel (a intercepté 4+ bugs avant test).
- Mémoire persistante : QWEN.md / CLAUDE.md (état, pièges, décisions).
- Jamais de secret en clair ; un PAT exposé = révocation immédiate.

3. Doctrine & vision produit

Interface minimaliste

- Surface type Claude Code/ChatGPT : une saisie, des chips, zéro surcharge.
- Divulgateion progressive : la richesse se déverrouille dans les Réglages.

Complexité backend

- Moteurs, services, agents dans Moto.Core / Snake2000.Engine.
- 100+ paramètres data-driven (1 ligne = 1 paramètre).

Fait maison

- Inspiration Zed, réécriture C# intégrale (zéro copie, Zed = GPL).
- Pas de dépendance runtime externe ; pas d'UI factice.

Séparation stricte des responsabilités

Entité	Rôle	Interdits
MOTO Editor	Édite, affiche, ouvre, sauvegarde, lance des commandes.	Parser profondément, générer une architecture, valider des namespaces, inventer des systèmes.
MOTO AI	Assiste, prédit, propose, route les demandes.	Appliquer sans confirmation ; sortir du local.
XENO-SSS∞	Opérations structurées projet complet : Scanner→Analyzer→Synthesizer→Connector→Validator.	—

Modèle d'appel : MOTO Editor appelle XENO comme Zed appelle Claude Code — l'éditeur *propose* et *affiche*, l'utilisateur (ou XENO) *applique*. Règle d'architecture vérifiée dans le code : « **MOTO propose, l'utilisateur/XENO applique** » (ex. InlineDiffPreviewService ne modifie jamais le document).

Échelle visuelle de référence (critère du fondateur)

Cheap
dépassé

Faible
dépassé

Moyen
en cours

Élevé
= bêta publique

Commercial
= v1.0 vendable

Références visuelles : Claude Code (effets, agencement menus) · Zen (menu IA, modularité explorateur) · VS Code (richesse fonctionnelle) · Zed (architecture).

4. Architecture & modules

Module	Contenu	État
Moto.Editor	Shell MAUI/WinUI 3 : 4 colonnes, explorateur, onglets+badges, terminal, status bar, palette, overlays, HomeView style Claude Code, MainPage.Panels (docks).	0 erreur
Moto.Core	374 fichiers : SettingsEngine/Catalog, moteurs cognitifs, IA (MotoAiKernel, Cortex, Neural), services (Git, Terminal, FileTree), sécurité, performance, agents.	0 erreur
Snake2000.Engine	XENO-SSS∞ : pipeline structuré 5 étapes ; délégation des opérations projet.	0 erreur
Shared/	Crypto & updates : Sha256Helper, Ed25519, UpdateManifest, PayloadVerifier, PayloadExtractor, ResumableDownloader, AtomicUpdater (non projet, réutilisé).	Stable

Flux de délégation (exemple : demande « gen/fix »)

Utilisateur → MOTO Editor (saisie) → MOTO AI (route, effort) → XENO-SSS∞
Scanner → Analyzer → Synthesizer → Connector → Validator
→ ProposedEdit / diff → MOTO Editor affiche (InlineDiffPreview) → utilisateur applique.

Types clés du contrat Editor↔IA (vérifiés dans le code)

Type	Rôle
ProposedEdit	Modification proposée (fichier, lignes, remplacement) — appliquée par l'Editor, jamais auto.
MultiBuffer / BufferExcerpt / BufferPosition	Vue virtuelle agrégeant plusieurs extraits de fichiers en buffer continu.
EffortMapper (0→4)	Profils Ultra rapide → Max (température, tokens, vérification) — déterministe, sans état.
DiffLine / DiffKind	Diff visuel avant application (Unchanged/Added/Removed).

5. Stack technique & contraintes

Couche	Choix	Justification / contrainte
Runtime	.NET 10 (net10.0-windows10.0.19041.0)	Machine équipée 9/10 ; .NET 8 absent au départ ; bascule documentée.
UI	MAUI + WinUI 3	Natif Windows, léger, sans Electron ; accès UI Automation (accessibilité).
Windows App SDK	1.8 épinglé	Seule version installée/testée ; changer = plantage déjà vécu.
Packaging	WindowsPackageType=None + MSIX optionnel	Deux exécutables : raccourci Bureau (unpackaged) et menu Démarrer (MSIX).
Éditeur de code	CodeEditorView = WebView maison	Gutter, coloration, minimap, scroll horizontal ; pas de composant tiers.
IA locale	Ollama + ONNX (qwen2.5-coder, qwen2.5-0.5b embarqué visé)	Zéro cloud ; décodage spéculatif planifié.
Mises à jour	AtomicUpdater + Ed25519 + CDN mirrors	Backup→extract→swap→rollback auto ; téléchargement résumable.
Dépendances	Aucune runtime externe	Doctrine « sans dépendance » ; NuGet débloqué par config locale projet.

Contraintes techniques assumées (objectivité)

Connues & documentées - Sous-système titlebar WinUI3/MAUI instable (barre bleue : 3 échecs SetBorderAndTitleBar). ~300 warnings de build (CS8618/CS8625/CS0618) — dette en cours de résorption. - Windows uniquement aujourd'hui ; macOS/Linux = PlatformEngine (planifié).	Choix de prudence - Pas de migration .NET/MAUI sous pression (échec .NET 10 du 30/08 sur ce projet). - Fichiers > 400 lignes jamais régénérés d'un bloc (risque de corruption silencieuse). - Chantiers isolés + revue croisée avant commit.
---	---

6. L'éditeur : cœur de métier

Shell 4 colonnes

- Explorateur de fichiers redimensionnable (DataTemplate 4 colonnes : indent, chevron, glyphe, nom).
- Onglets + badges d'état ; recherche par nom de fichier en direct.
- StatusBar : état, erreurs, monitoring, dashboard performance (item 107).
- Palette de commandes Ctrl+Shift+P (ranking maison).

CodeEditorView (WebView maison)

- Gutter + numéros de ligne ; coloration syntaxique incrémentale.
- Mini-map compressée ; scroll horizontal.
- Ghost text (Tab) issu de l'IA locale.
- Undo 1000 niveaux via PieceTableBuffer (Tier 30).

Accueil style Claude Code

Élément	Description livrée
Barre de saisie	« Décris une tâche ou pose une question » + flèche animée ; route vers MOTO AI.
Chips	Local (menu overlay au-dessus), Rechercher projet (sélecteur de dossier), dossier+.
Barre basse	+ / micro / IA / Cortex · sélecteur modèle (MOTO AI par défaut) · Puissance (Ultra...) · anneau budget « Token : infini ».
Stats	Sessions / Messages / Tokens / Patterns appris — compactées.

Terminal & dock du bas (livrés)

Dock du bas avec **terminal interactif réel** : TerminalService (shell cmd.exe, sortie en direct, envoi de commandes), replié par défaut, ouverture Ctrl+`. Onglets limités aux backends réels (doctrine « pas d'UI factice »). Complété par Output/Diagnostics lorsque leurs services existeront.

7. Panneaux modulaires façon Zed (5 briques livrées)

#	Brique	Comportement	Réf. commit
1	Redimensionnement souris	Tous docks/panneaux redimensionnables au bord.	728664f
2	Changement de côté	Bouton dédié ; docks gauche/droite interchangeables.	livré
3	Réordonnancement DnD	Glisser-déposer des panneaux dans un même dock.	livré
4	DnD inter-docks	Panneau IA → dock droit (empilé sous explorateur/sessions, zone d'accueil à la demande) ; préférence d'hôte conservée.	ef8bdeb
5	Dock du bas	Terminal réel + futures sorties (Output/Diagnostics).	1b7a830

Panneaux IA concernés : **Cortex, Neural, Workspace, Galerie, Analytics, Debug, Plateforme**. La revue croisée a intercepté avant test une régression sérieuse (bouton « changer de côté » cassé par la restructuration) + 3 bugs discrets — corrigés dans le même commit.

Ce que la modularité apporte (vs concurrence)

Vs VS Code

- Pas de docks natifs : panneaux groupés manuels.
- MOTO : 3 docks + DnD libre natifs.

Vs Zed

- 3 docks mais disposition par défaut imposée (critiquée).
- MOTO : disposition au choix, persistable (à finaliser).

Vs JetBrains

- Panneaux ancrés + « perspectives » figées.
- MOTO : agencement libre par tâche.

8. IA intégrée & agents spécialisés

Noyau local

- **MotoAiKernel** : orchestration locale (Ollama + ONNX), 4 modes Eco/Balanced/Turbo/Ultra.
- **Cortex** : mémoire + style de l'utilisateur ; suggestions proactives.
- **Neural** : embeddings locaux, recherche sémantique projet.
- **ProjectUnderstandingEngine** : compréhension structurelle légère.
- **TimeMachine** : instantanés / retour arrière de sessions.

Surface utilisateur

- Chat multi-threads dockable ; ghost text (Tab) ; Explain Everything.
- Sélecteur de modèle + « Puissance » (EffortMapper 0→4 : Ultra rapide → Max).
- Preview diff inline avant application (règle : jamais auto-appliqué).
- Explainability logs : chaque suggestion tracée (audit).

20 agents spécialisés (cadre SpecializedAgentRegistry)

Exemple d'agent	Priorité / coût	Rôle
agent.tests.micro / TestGeneratorAgent	P1 · CPU 0.3	Génère des tests ciblés sur le diff.
agent.security.hint	P1	Signale patterns à risque (secrets, injections).
agent.code.health / SearchSummarizerAgent	P1	Santé du code ; synthèse de recherche projet.
ChangelogAgent / CommitMessageAgent	P2	Notes de version & messages de commit conventionnels.
DocEngine (6 markdown auto)	P2	Documentation générée et maintenue.

Architecture vérifiée : les agents **assistants** tournent sur MotoAiKernel local ; l'analyse profonde / génération projet est déléguée à XENO-SSS∞. Identifiants d'agents vérifiés un par un dans le dépôt.

9. Moteurs internes (catalogue)

Moteur / module	Fonction	Statut
PatternDetectorEngine	Détection de patterns d'usage (« Patterns appris »).	Livré
CodeImprovementEngine	Suggestions d'amélioration ciblées.	Livré
UserHistoryEngine	Historique d'actions → proactivité.	Livré
PedagogyEngine	Mode pédagogique / explications pas-à-pas.	Livré
NavigationAssistantEngine	Assistance à la navigation projet.	Livré
ArchitectureBuilderEngine	Lecture structurelle (déléguée à XENO pour écriture).	Livré
UiGeneratorEngine	Génération d'UI locale (cadre).	Livré
ContextEngine (double hook)	Contexte actif pour l'IA (fichier, sélection, projet).	Livré
AutoLink / Builders / Cortex / Neural	Liens auto, constructeurs, mémoire, embeddings.	Livrés
ConsistencyScanner	Fichiers/projets « fantômes », deps inutilisées.	Noté (à brancher)
InconsistencyDetector / FileIntegrity	Intégrité fichiers (hash, reconstruction).	Livré
StructuredLogCollector	Logs JSON structurés + Explainability.	Livré

Répartition Editor / AI / XENO (rappel doctrine)

Editor = afficher, éditer, ouvrir, sauvegarder, commander (jamais d'analyse profonde)
 AI = assister, prédire, proposer, router (jamais d'auto-application)
 XENO = scanner, analyser, synthétiser, connecter, valider (opérations projet)

10. Sécurité, mises à jour & privacy

Mécanisme	Description (vérifiée dans Shared/)	Niveau
Signature Ed25519	update.priv (secret, jamais committé) / update.pub embarqué (BuildKeys.cs) ; vérification payload + manifeste.	Fort
SHA256 double	Par fichier + payload global dans payload.json.	Fort
AtomicUpdater	Backup → extraction temporaire → swap → rollback automatique en cas d'échec.	Fort
ResumableDownloader	HTTP Range (reprise) + mirrors CDN fallback.	Solide
Anti-ZipSlip	Extraction contrôlée des chemins (PayloadExtractor).	Fort
IsolatedInferenceHost + sandbox	Inférence LLM isolée ; sandbox chemins/URL pour outils.	Solide
PrivacyScanner	Détection de fuites de données sensibles dans les sorties IA.	Solide

Privacy by design

Garanties

- 0 octet de code utilisateur envoyé à l'extérieur (IA locale).
- Aucune télémétrie ; opt-in uniquement si ajouté un jour.
- RGPD : quasi aucune donnée collectée → politique courte et vraie.

Gestion des secrets (règle interne)

- Jamais de PAT/clé en clair ; incident passé = révocation immédiate.
- Captures publiées inspectées (aucun chemin/identité visible).

11. Performance & optimisations

Composant	Effet	Statut
4 modes Eco / Balanced / Turbo / Ultra	Arbitre qualité IA vs ressources machine.	Livré
LazyFileLoader / PerFileOnDemandService	Chargement paresseux des gros projets.	Livré
IncrementalHighlighter	Coloration incrémentale (pas de reparse global).	Livré
MiniMapCompressor	Mini-map à coût mémoire réduit.	Livré
AiCacheEngine	Cache des réponses IA (latence & coût locaux).	Livré
PieceTableBuffer (Tier 30)	Buffer de texte optimisé, undo 1000.	Livré
PerformanceProfiler + StatusBar	Mémoire/CPU/fps/mode throttlés (1 s).	Livré
CI perf gate + nightly benchmarks	Tolérance 15 % vs baseline ; audit 150 Mo.	Workflow livré

Objectifs mesurés (config de référence)

< 1 s

DÉMARRAGE À FROID (VISÉ)

< 300 Mo

EMPREINTE MÉMOIRE (VISÉE)

0

OCTET ENVOYÉ AU CLOUD

1000

NIVEAUX D'UNDO

Honnêteté : la latence IA dépend du modèle local (qwen2.5-coder:7b mesuré lent sur machine modeste → recommandation d'un modèle 3B en mode Eco). Le rendu GPU type GPU de Zed n'est pas égalé : choix assumé du natif MAUI plutôt que d'un framework maison.

12. Réglages : 100+ paramètres data-driven

Le système de réglages est le véritable tableau de bord de la complexité : chaque fonctionnalité est un paramètre déclaratif (1 ligne = 1 paramètre, SettingItem<T>), recherché, templatisé et persisté. La fenêtre Réglages suit le modèle Zed : 15 catégories.

Catégories (15)

- Général · Apparence · Keymap · Éditeur · Langues & Outils
- Recherche & Fichiers · Fenêtre & Layout · Panneaux
- Debugger · Terminal · Version Control · Collaboration
- IA · Réseau · Développeur

Capacités

- Recherche instantanée dans les paramètres.
- Templates de configuration.
- Persistence au redémarrage.
- Exposition progressive (97 exposés, ~300 au catalogue).

Exemples de familles

- terminal.* (shell, police, scrollbar, départ replié)
- docks.* (hauteurs, visibilité, côtés)
- ai.* (modèle, effort, ghost text, explicabilité)
- perf.* (mode Eco→Ultra, caches)

Pourquoi c'est un avantage concurrentiel

Concurrent	Modèle de personnalisation	Limite
VS Code	settings.json + ~50 000 extensions	Chaos, instabilité, surcharge.
Zed	JSON unique	Puissant mais abrupt ; dispositions imposées.
JetBrains	Dialogues riches	Verrouillé, peu extensible.
MOTO	Catalogue data-driven + UI minimaliste	Profondeur sans chaos ; divulgation progressive.

13. Collaboration, remote, export & documentation

Collaboration & remote

- Collab temps réel CRDT-light (CollabSession / PatchEngine) ; CRDT réel = v1.0.
- Remote WS + SSH (édition à distance).
- Presence-aware suggestions (évite l'IA lourde en co-édition).
- GitHub OAuth device flow connecté (import/publication à finaliser).

Export & livrables

- Export 9 formats ; Présentations auto (pitch / slides / estimation).
- Doc Engine : 6 markdown générés et maintenus.
- Time Machine : instantanés de session rejouables.
- Health : rapport de santé projet (bouton « à venir » assumé).

Git & outillage développeur

Fonction	État
GitService (CLI git) : statut, commits, branches	Livré
Diff inline + preview avant application	Livré
CommitMessageAgent / ChangelogAgent	Livrés
Branches visuelles, blame, historique graphique	À venir (v1.0)

14. Design & identité visuelle

Palette officielle (MotoTheme.xaml)

#111214 Fond dark	#1B1C1F Surface	#007ACC Accent	#E8EAED Texte
#F7F8FA Fond light	#FFFFFF Surface light	#0066CC Accent light	#33353A Bordures

Style « Hybride Claude »

- Arrondis modérés, espacement généreux, ombres discrètes sur flottants.
- Icônes Segoe Fluent Icons vérifiées à l'écran (jamais de case vide).
- Menus déroulants en overlay au-dessus de leur bouton (ne poussent pas le layout).
- Dégradé doux + halo discret sur l'accueil ; animations ciblées (flèche d'envoi).

Identité

- Logo hexagone « M » bleu ; baseline « Léger. Rapide. Sans dépendance. Tout fait maison. »
- Typo : Segoe UI / Inter ; titres en sentence case.
- Cohérence dark/light via DynamicResource (thème clair non activé = choix assumé, pas un oubli).

Méthode visuelle : chaque élément passe par l'échelle cheap → faible → moyen → élevé → commercial ; rien n'est publié sous « moyen ». Captures publiées sans donnée personnelle, thème dark par défaut.

15. Marché & comparaison concurrentielle

16,5 Md\$

MARCHÉ IDE 2026 (CAGR 10,8 %)

84 %

DEVS UTILISANT/PLANIFIANT L'IA

75,9 %

PART D'USAGE VS CODE

1,7→7,3 %

ZED EN 1 AN (PREUVE D'OUVERTURE)

Caractéristique	MOTO Editor	Zed	VS Code	JetBrains	Cursor
Technologie	C# / MAUI / WinUI3	Rust / GPUI GPU	JS / Electron	Java	Fork VS Code
Cloud	Aucun (local-first)	Optionnel	Oui (Copilot)	Oui	Oui (cœur)
Docks modulaires	3 docks + DnD	3 docks	Panneaux	Panneaux	3 docks
Terminal réel	Oui	Oui	Oui	Oui	Oui
IA	XENO + locale	Claude Code	Copilot	Natif	Agent natif
Personnalisation	100+ params UI	JSON	Extensions	Thèmes	Limitée
Accessibilité	UI Automation (potentiel fort)	Défi GPUI	Bonne	Bonne	Débutante
Prix	Gratuit → Pro 10-20 €	Pro \$10 / +\$60	Gratuit	\$25-50	Pro \$20 / U \$200

Lecture objective : MOTO ne bat Zed ni sur le rendu GPU ni sur la maturité ; il se distingue sur **local-first/privacy**, **personnalisation sans chaos** et **accessibilité native** — trois niches mal servies.

16. SWOT & positionnement

Forces

- 100 % local, léger, sans Electron ; sécurité signée Ed25519.
- Architecture modulaire Zed livrée (5 briques) + terminal réel.
- ~300 paramètres data-driven ; IA locale + 20 agents.
- Méthode transparente (revues croisées, mémoire QWEN/CLAUDE.md).

Faiblesses

- Notoriété nulle ; équipe d'une personne ; Windows uniquement.
- ~300 warnings ; barre bleue non résolue ; LSP/CRDT/DAP absents.
- Dépendance à la stack MAUI/WinUI3 (régressions possibles).

Opportunités

- Marché +10,8 %/an ; 84 % de devs vers l'IA ; fatigue VS Code.
- Niches privacy & accessibilité sous-servies ; modèle freemium prouvé.
- Histoire « 1 humain + 3 IA » = actif marketing organique.

Menaces

- VS Code 75,9 % ; entrants financés (Cursor, Windsurf) ; Zed rapide.
- Adoption entreprise lente ; fragmentation des alternatives.

Positionnement retenu

Ne pas attaquer VS Code frontalement. Créer et dominer la catégorie « IDE minimaliste, local & accessible » : façade simple type Claude Code, profondeur Zed, confidentialité totale. Objectif prudent : 5-10 % de part de marché à 5 ans (trajectoire Zed comme référence).

17. Roadmap & modèle économique

Versions livrées / à venir (FEATURES.md & Roadmap)

Version	Contenu	Statut
v0.1 Fondation	Shell 4 colonnes, explorateur, onglets, terminal, status bar, CodeEditorView, SettingsEngine, XENO base.	Livrée
v0.2 UI moderne	Layout Zen, chat dockable, ghost text, export 9 formats, présentations, remote WS+SSH, collab CRDT-light, Time Machine/Health/Pattern/AutoLink/Context/Doc/Perf.	Livrée
v0.3 IA interne	Moteurs cognitifs, agents, panneaux modulaires Zed (5 briques), dock bas + terminal réel.	Livrée
v1.0	LSP Roslyn · CRDT réel · DAP debugger · tests · i18n · accessibilité · barre de titre résolue · persistance disposition.	À venir

Horizons d'exécution

H1 (sept.-oct. 26)

- Barre de titre (chantier dédié, sondes).
- Persistance docks ; 100+ params exposés ; warnings < 50.

H2 (oct. 26-janv. 27)

- MotoBridge/Tier 28 (XENO ↔ Editor, diffs affichés).
- LSP Roslyn ; LLM embarqué ONNX ; marketplace agents.

H3 (janv.-juin 27)

- PlatformEngine macOS/Linux ; installateurs signés + winget.
- CRDT réel + DAP ; i18n ; accessibilité → v1.0 publique.

Modèle économique (bootstrap)

Source	Prix	Démarrage
Gratuit (cœur open source, GPL-3.0 visé)	0 €	Immédiat (adoption, confiance)
Pro (builds nightly signés, support prioritaire, agents premium)	10 €/mois · 96 €/an	Lancement v1.0
Dons (GitHub Sponsors / Ko-fi)	Libre	Dès l'open-sourcing
Marketplace (commission) + support B2B	20 % / ~500 €/an	An 2-3

18. Limites honnêtes, méthode & conclusion

Ce qui n'est pas encore vrai aujourd'hui (transparence)

Sujet	État réel
Barre de titre bleue Windows	Non résolue ; 3 tentatives SetBorderAndTitleBar échouées ; chantier dédié planifié (H1).
Multiplateforme	Windows uniquement ; PlatformEngine = H3.
LSP / DAP / CRDT réel / tests / i18n / accessibilité	Annoncés « à venir v1.0 », non livrés.
Boutons « Santé du projet / Snapshot »	Présents en UI, backend partiel — assumé comme « à venir », pas caché.
Dette	~300 warnings ; fichiers legacy en cours de nettoyage.

Pourquoi cette transparence est un actif

Chaque limite ci-dessus est **publique** (roadmap, tickets, devlog). Dans un marché où la confiance est le premier critère d'adoption d'un outil de code, « dire vrai, dire mesuré, dire utile » (charte éditoriale) remplace le budget marketing : les échecs documentés (barre bleue, échec .NET 10 du 30/08) sont publiés tels quels.

Conclusion

MOTO Editor est, à ce jour, un **IDE local-first fonctionnel et cohérent** : shell modulaire façon Zed complet, terminal réel, IA locale orchestrée, sécurité de niveau professionnel, et un système de réglages qui porte la complexité promise (100+ paramètres). Il n'est ni plus rapide que Zed, ni plus riche que VS Code — et ne cherche pas à l'être : il vise la niche **minimalisme + local + accessibilité**, avec une méthode de développement (1 humain + 3 IA, revues croisées, mémoire persistante) qui compense l'absence d'équipe. Le passage en bêta publique est conditionné à l'atteinte du niveau visuel « élevé » et à la v1.0 fonctionnelle (LSP, CRDT réel, DAP, tests, i18n, accessibilité).

0 ERREUR BUILD · 3 PROJETS	5/5 BRIQUES PANNEAUX ZED	100 % LOCAL · 0 CLOUD	v1.0 CAP 2027 (JUIN)
-------------------------------	-----------------------------	--------------------------	-------------------------